WEEK -2

1.**write a program for congestion control using Leaky bucket algorithm**.

```
import time
import random

bucket_capacity = 25
outflow_rate = 5
bucket_content = 0
incoming_packets = [30, 8, 8, 12, 8, 0]

print(f"Starting Simple Leaky Bucket Simulation...")
print(f"Capacity: {bucket_capacity} | Leak Rate: {outflow_rate}\n")

for i, packet in enumerate(incoming_packets, 1):
    print(f"--- Step {i} ---")
    print(incoming packet: {packet} units")

    if bucket_content + packet <= bucket_capacity:
        bucket_content += packet
        print(f"Accepted! Bucket content: {bucket_content}/{bucket_capacity}")
    else:
        dropped = (bucket_content + packet) - bucket_capacity
        bucket_content = bucket_capacity
        print(f"Overflow! Dropped {dropped} units. Bucket is full:
{bucket_content}/{bucket_capacity}")

    transmitted = min(bucket_content, outflow_rate)
    bucket_content -= transmitted
    print(f"Transmitted: {transmitted} units. Remaining in bucket: {bucket_content}\n")
```

```
    time.sleep(0.5)

print("--- Draining Remaining Data ---")
while bucket_content > 0:
    transmitted = min(bucket_content, outflow_rate)
    bucket_content -= transmitted
    print(f"Transmitted: {transmitted} units. Remaining in bucket: {bucket_content}")
    time. Sleep(0.5)

print("Simulation complete!")
```

Output:

```
Starting Simple Leaky Bucket Simulation...
Capacity: 25 | Leak Rate: 5

--- Step 1 ---
Incoming packet: 30 units
Overflow! Dropped 5 units. Bucket is full: 25/25
Transmitted: 5 units. Remaining in bucket: 20

--- Step 2 ---
Incoming packet: 8 units
Overflow! Dropped 3 units. Bucket is full: 25/25
Transmitted: 5 units. Remaining in bucket: 20

--- Step 3 ---
Incoming packet: 8 units
Overflow! Dropped 3 units. Bucket is full: 25/25
Transmitted: 5 units. Remaining in bucket: 20
```

```
--- Step 4 ---
Incoming packet: 12 units
Overflow! Dropped 7 units. Bucket is full: 25/25
Transmitted: 5 units. Remaining in bucket: 20

--- Step 5 ---
Incoming packet: 8 units
Overflow! Dropped 3 units. Bucket is full: 25/25
Transmitted: 5 units. Remaining in bucket: 20

--- Step 6 ---
Incoming packet: 0 units
Accepted! Bucket content: 20/25
Transmitted: 5 units. Remaining in bucket: 15

--- Draining Remaining Data ---
Transmitted: 5 units. Remaining in bucket: 10
Transmitted: 5 units. Remaining in bucket: 5
Transmitted: 5 units. Remaining in bucket: 0
Simulation complete!
```

2. using TCP/IP sockets , write a client -server program to make client sending the file name and the server to send back the contents of the requested file if present.

TCP server:

```
import socket
import os
server = socket.socket()
server.bind(("127.0.0.1", 6001))
server.listen(1)

print("TCP Server started")
print("Waiting for client...")

conn, addr = server.accept()
print("Client connected")

filename = conn.recv(1024).decode()

if os.path.isfile(filename):
    size = os.path.getsize(filename)
    conn.send(str(size).encode())

    with open(filename, "rb") as f:
        while True:
            data = f.read(1024)
            if not data:
                break
            conn.sendall(data)

    print("File sent successfully")
else:
```

```
    conn.send(b"0")
    print("File not found")

conn.close()
server.close()
```

Output:

```
TCP File Transfer Server Started...
Waiting for client...

Client connected successfully.
Requested file: test.txt

File found.
File size: 27 bytes
Sending file...

File sent successfully.
Connection closed.
```

TCP client:

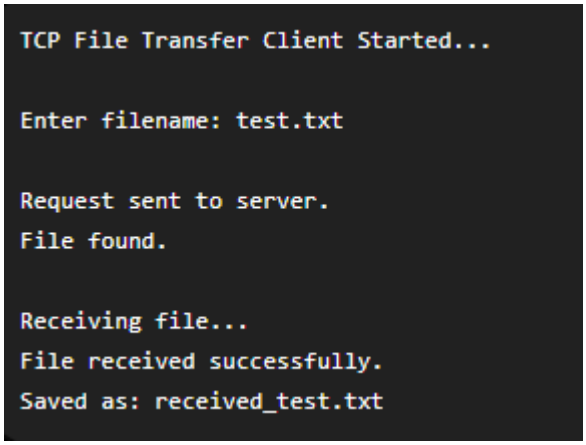
```
import socket
client = socket.socket()
client.connect(("127.0.0.1", 6001))

filename = input("Enter filename: ")
client.send(filename.encode())
size = int(client.recv(1024).decode())

if size > 0:
    with open("received_" + filename, "wb") as f:
        while size > 0:
            data = client.recv(min(1024, size))
            f.write(data)
```

```
            size -= len(data)
    print("File received successfully")
else:
    print("File not found")
client.close()
```

Output:



3. using UDP sockets , write a client -server program to make client sending the file name and the server to send back the contents of the requested file if present.

UDP server:

```
import socket
import os
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(("127.0.0.1", 6002))

print("UDP Server started")
print("Waiting for client...")

data, addr = server.recvfrom(1024)
filename = data.decode()

if os.path.isfile(filename):
    size = os.path.getsize(filename)
    server.sendto(("FOUND " + str(size)).encode(), addr)

    with open(filename, "rb") as f:
        while True:
            data = f.read(1024)
            if not data:
                break
            server.sendto(data, addr)

    print("File sent successfully")
else:
    server.sendto(b"NOT_FOUND", addr)
    print("File not found")
```

server.close()

Output:

```
UDP File Transfer Server Started...
Waiting for client...

Requested file: test.txt
File found.

Sending file...
File sent successfully.
Server stopped.
```

UDP client:

```
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

filename = input("Enter filename: ")
client.sendto(filename.encode(), ("127.0.0.1", 6002))

data, addr = client.recvfrom(1024)
response = data.decode()

if response.startswith("FOUND"):
    size = int(response.split()[1])

    with open("received_" + filename, "wb") as f:
        while size > 0:
            data, addr = client.recvfrom(1024)
            f.write(data)
            size -= len(data)
    print("File received successfully")
else:
    print("File not found")
```

client.close()

Output:

```
UDP File Transfer Client Started...

Enter filename: test.txt

Request sent to server.
File found.

Receiving file...
File received successfully.
Saved as: received_test.txt
```